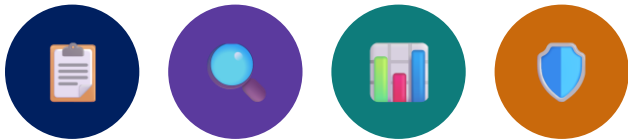


## MODULE 20

# Logging Frameworks and Diagnostics

Explore enterprise-grade logging frameworks, structured logging, and diagnostics techniques for reliable production systems.





# MODULE 20 OVERVIEW

*Logs are one major observability signal. They complement metrics and traces by providing detailed event context.*

- This module explores enterprise-grade logging frameworks, structured logging, and diagnostics techniques to improve observability, troubleshoot issues, and ensure reliable production systems.

## DURATION & LAB



### Theory

SLF4J, Logback, log levels, structured logging.



### Lab

Structured Logging in a Spring Boot app.



### Scenario

Add production-ready structured logs with correlation IDs.

## BUILD

SLF4J + Logback structured logging

## OBSERVE & DIAGNOSE

Correlation IDs, exceptions, centralized logs

## THE PAYOFF

Faster troubleshooting, reliable systems

**KEY TAKEAWAY** You can't improve what you can't see — logs turn data into actionable insights.

# WHY LOGGING MATTERS

*Logs are the primary window into what your application is actually doing in production.*



## Visibility

See what's really happening inside a running application.



## Faster Diagnosis

Pinpoint root causes without reproducing the issue live.



## Monitoring

Feed dashboards and alerts that catch problems early.



## Auditability

Provide a durable record for security and compliance reviews.



## Request Correlation

Connect related log events across services using correlation or trace context.



## Faster Recovery

Shorten mean time to resolution during incidents.

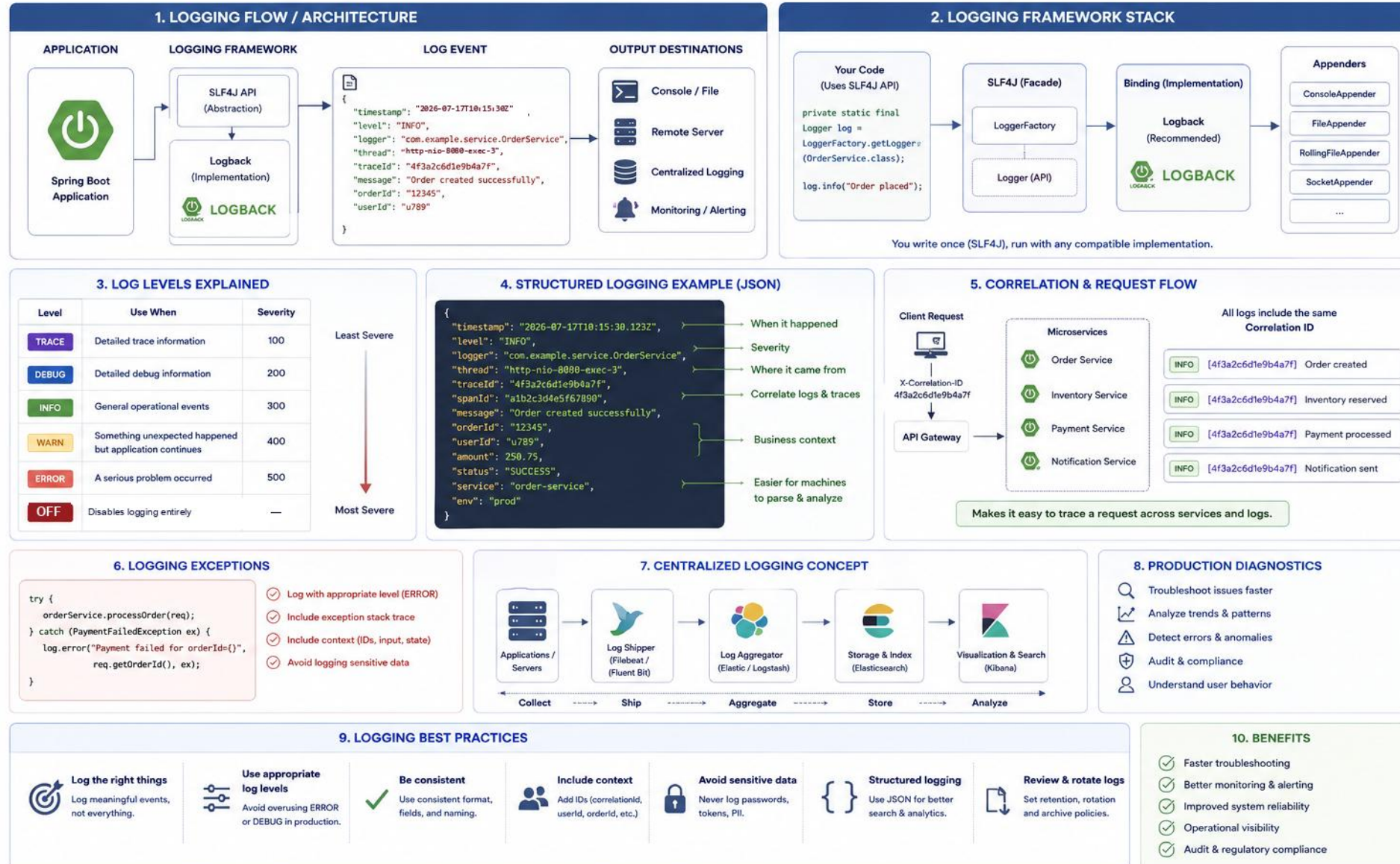
*Good logs. Better insights. Reliable systems. Log data also supports security/compliance audits and informed operational decisions. Real distributed tracing (trace ID, span ID, parent-child spans, service graph) is covered in Module 21.*

**KEY TAKEAWAY** You can't improve what you can't see — logs turn data into actionable insights.



# LOGGING IN MODERN APPLICATIONS

Capture the right information at the right level to understand, troubleshoot and improve your system.



# LOGGING VS DEBUGGING

*Debugging and logging solve different problems — one is for development, the other for production.*

## DEBUGGING

- Interactive, step-by-step inspection
- Requires attaching a debugger to the process
- Only practical in development environments
- Great for deep-diving a single reproducible issue

VS

## LOGGING

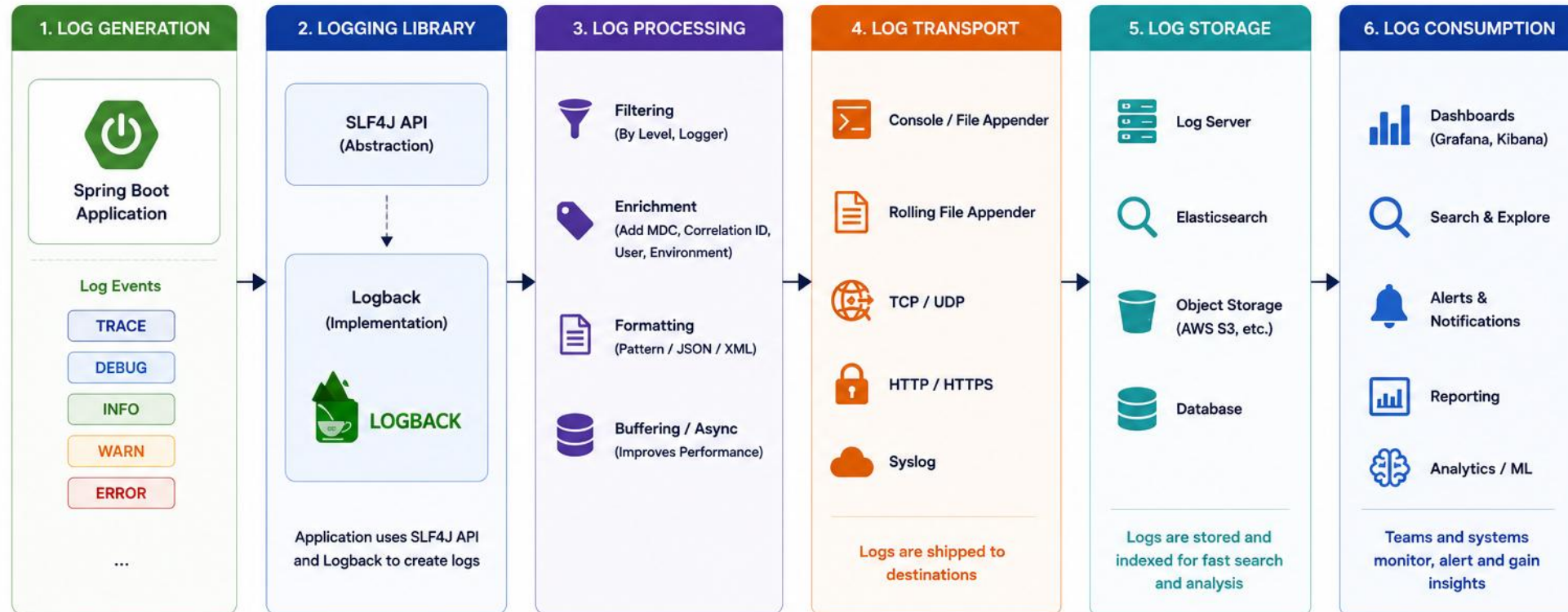
- Passive, always-on record of events
- No special tooling needed to capture
- Works everywhere, including production
- Great for understanding behavior over time and across requests

*Interactive debugging is generally unsuitable for normal production troubleshooting because it can pause execution, alter timing, and create security risk. Logs and production-safe diagnostics (diagnostic snapshots, tightly-controlled remote debugging, Java Flight Recorder, heap/thread dumps, profilers, debug containers) are the default and the exception, respectively. Logging can be production-safe when levels, volume, formatting, and transport are designed carefully.*

**KEY TAKEAWAY** Debugging pauses execution and doesn't scale to production — logs and production-safe diagnostics are the default tools there.

# LOGGING ARCHITECTURE

High-level view of how logs are generated, processed, transported, stored and consumed.



# ENTERPRISE LOGGING PIPELINE

*Application → SLF4J → Logback/Appender → Collector → Storage/Index → Search/Dashboard/Alert — one pipeline; actual tooling varies by organization.*

| STAGE                      | EXAMPLE TOOLS                        | PURPOSE                            |
|----------------------------|--------------------------------------|------------------------------------|
| Application                | Application / service code           | Generate log events                |
| SLF4J                      | Logging facade API                   | Format and handle events           |
| Logback / Appender         | Logging backend + appender           | Send events to destinations        |
| Collector                  | Agent / collector                    | Collect logs from many sources     |
| Storage / Index            | Searchable storage / archive storage | Index logs for search & retention  |
| Search / Dashboard / Alert | Dashboard / alert destination        | Turn logs into dashboards & alerts |

| LAYER                             | TYPICAL OWNER         |
|-----------------------------------|-----------------------|
| Log event design                  | Application team      |
| Application logging configuration | Application/platform  |
| Collection and transport          | Platform/SRE          |
| Storage and retention             | Platform/security     |
| Dashboards and alerts             | SRE/application teams |

- Categories are illustrative — actual tooling varies by org. Example stack: SLF4J → Logback → Fluent Bit → Kafka → OpenSearch → Kibana.
- Async appenders cut request-thread blocking for high-volume systems but add queueing, a dropping policy, shutdown flushing, memory use, and ordering considerations.

**KEY TAKEAWAY** Real logging pipelines chain multiple specialized tools — producers to dashboards — into one observability platform.



# INTRODUCTION TO SLF4J

*SLF4J (Simple Logging Facade for Java) is an abstraction layer over concrete logging frameworks.*

## EXAMPLE

```
private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

log.info("Order {} placed for user {}",
    orderId, userId);

if (log.isDebugEnabled()) {
    log.debug("Customer snapshot {}", expensiveSnapshot());
}
```

## WHY A FACADE?

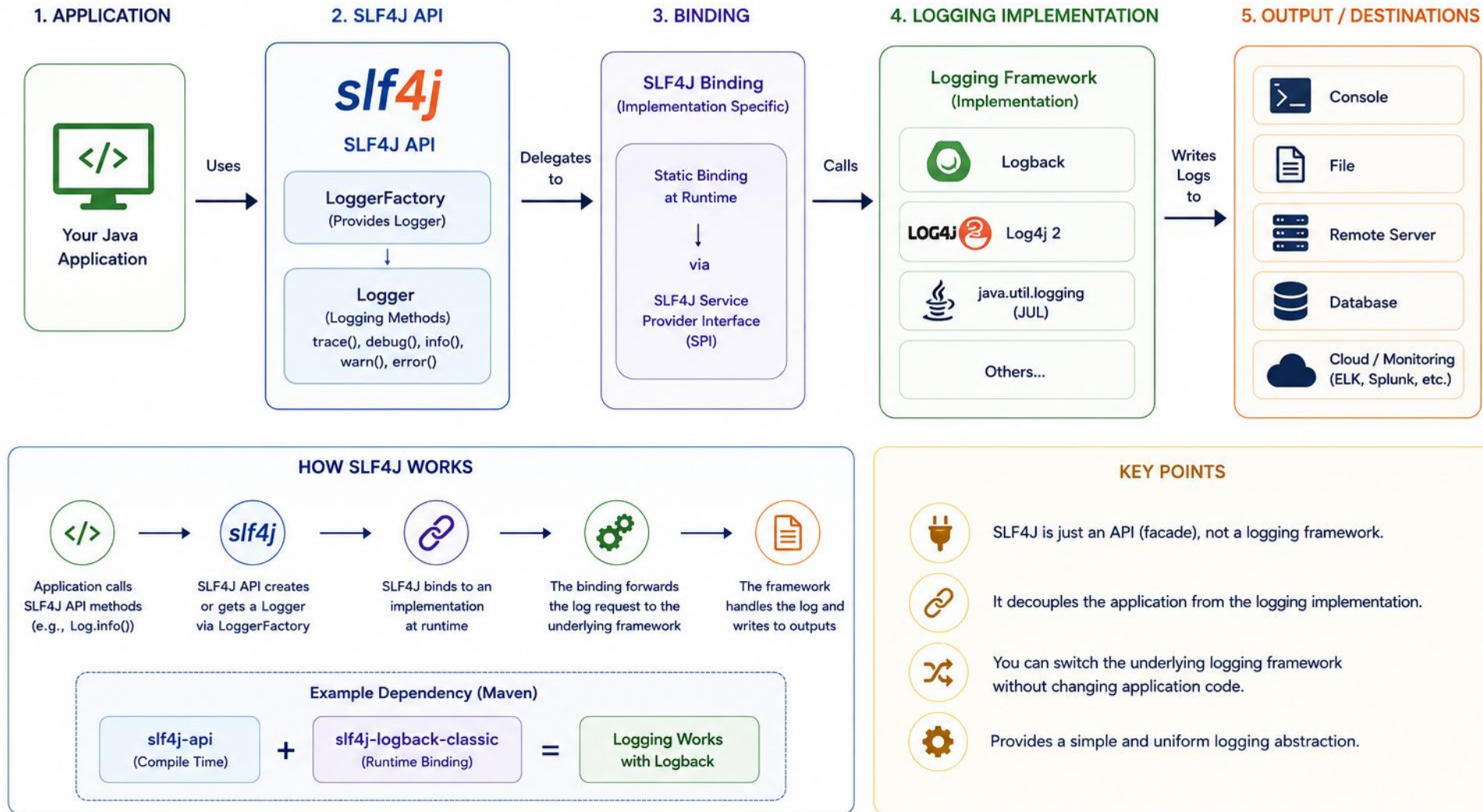
- Decouples your code from a specific logging implementation.
- SLF4J keeps application code independent of the backend API, making implementation changes easier without rewriting logging calls.
- Provides efficient parameterized logging (no string concatenation).
- The de facto standard logging API for Java.
- Keep one active SLF4J provider on the runtime classpath — multiple providers or logging bridges can cause warnings, duplicate events, or recursion.

*"Write once, log anywhere." Common bindings: Logback (default), Log4j 2, java.util.logging, and others. Placeholders reduce unnecessary string construction; guard genuinely expensive arguments with a level check like `isDebugEnabled()` — use sparingly, not for every call.*

**KEY TAKEAWAY** Code against the SLF4J API, not a specific logging framework — it keeps your logging backend swappable.

# SLF4J OVERVIEW

SLF4J (Simple Logging Facade for Java) is a façade or abstraction for various logging frameworks. It allows applications to remain independent of the underlying logging implementation.



# INTRODUCTION TO LOGBACK

*Logback is the most common SLF4J implementation, and the default in Spring Boot.*

## EXAMPLE: logback-spring.xml

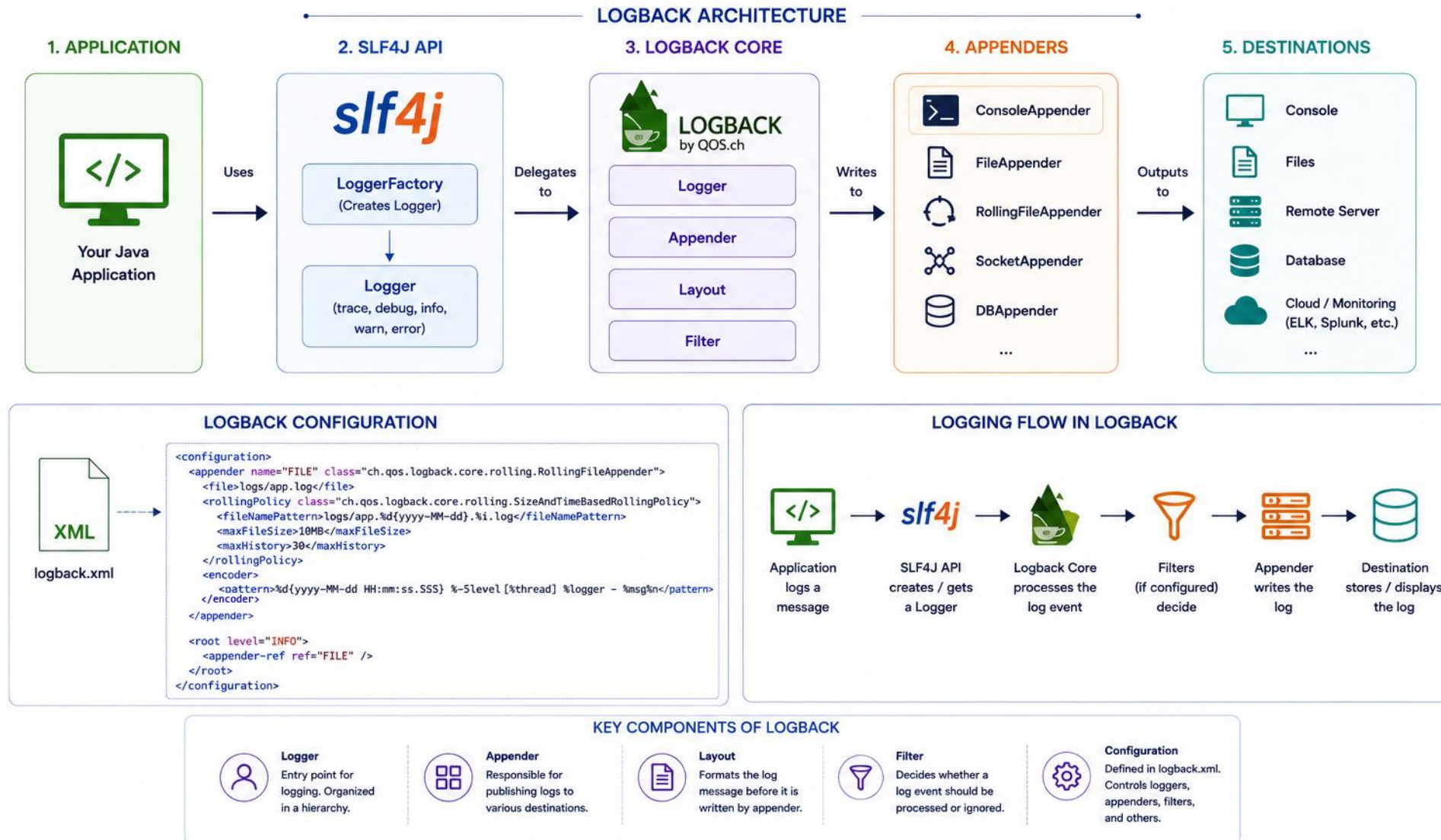
```
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{ISO8601} %-5level [%thread] %logger{36} corr=%X{correlationId:-none} - %msg%n</pattern>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="CONSOLE"/>
  </root>
</configuration>
```

*Logback is a mature SLF4J backend and the default logging implementation in many Spring Boot versions. Log4j 2 is another modern alternative. Console or file output collected asynchronously by the platform is common — direct network, database, or email appenders require careful reliability and performance design.*

**KEY TAKEAWAY** logback.xml (or logback-spring.xml) configures appenders, patterns, and log levels for your whole application.

# LOGBACK OVERVIEW

Logback is a powerful, high-performance logging framework for Java.  
It is a successor of Log4j and a native implementation of **SLF4J**.





# LOG LEVELS EXPLAINED

*Log levels let you control verbosity and filter logs by severity.*

| LEVEL | MEANING                                                                                     | WHEN TO USE                             |
|-------|---------------------------------------------------------------------------------------------|-----------------------------------------|
| ERROR | Unexpected failure requiring investigation, or a failure causing the request/system to fail | Exceptions, failed operations           |
| WARN  | Abnormal condition handled successfully but worth monitoring                                | Deprecated usage, fallback behavior     |
| INFO  | Meaningful lifecycle or business event useful in normal operations                          | Startup, request received, order placed |
| DEBUG | Diagnostic detail for troubleshooting                                                       | Local development, deep troubleshooting |
| TRACE | Highly granular flow detail                                                                 | Rarely enabled, very deep debugging     |

- Do not log every 4xx as an application error — use metrics for volume, security audit logs where appropriate, and reserve stack traces for unexpected defects.
- Use INFO for low-volume, operationally significant events; high-volume request details may belong in access logs, structured telemetry, sampled logs, or DEBUG depending on policy.

**KEY TAKEAWAY** Use the right level for the right audience — INFO for operations, DEBUG for developers, ERROR for incidents.

# LOG LEVELS IN PRACTICE

*The same order-processing scenario shows why each log level exists.*

## EXAMPLE

```
log.trace("Entering processOrder id={}", orderId);
log.debug("Discount rule {} applied to order {}",
    ruleId, orderId);

log.info("Order {} placed for user {}", orderId, userId);
log.warn("Retrying payment for order {} (attempt {})",
    orderId, attempt);

log.error("Order {} failed: gateway timeout", orderId, ex);
```

*Same event, five severities — the log statement's position in the code doesn't set the level; the operational meaning of what happened does. This mirrors the Log Levels Explained table on the previous slide.*

## MAPPING LEVELS TO THIS SCENARIO

- TRACE — very granular step tracking, rarely enabled outside deep debugging.
- DEBUG — internal decision detail (e.g., which discount rule fired) for developers.
- INFO — a meaningful business event: the order was placed.
- WARN — an anomaly the code handled itself, like a payment retry that succeeded.
- ERROR — pass the exception object last; SLF4J attaches the full stack trace automatically.

**KEY TAKEAWAY** Pick the level by asking who needs to see this and how urgently — not by how deep the code sits.

# TRY IT YOURSELF — EXERCISE 1: LOG LEVEL QUIZ

*Reinforces: choosing DEBUG / INFO / WARN / ERROR for real CRM events before Lab 20.*

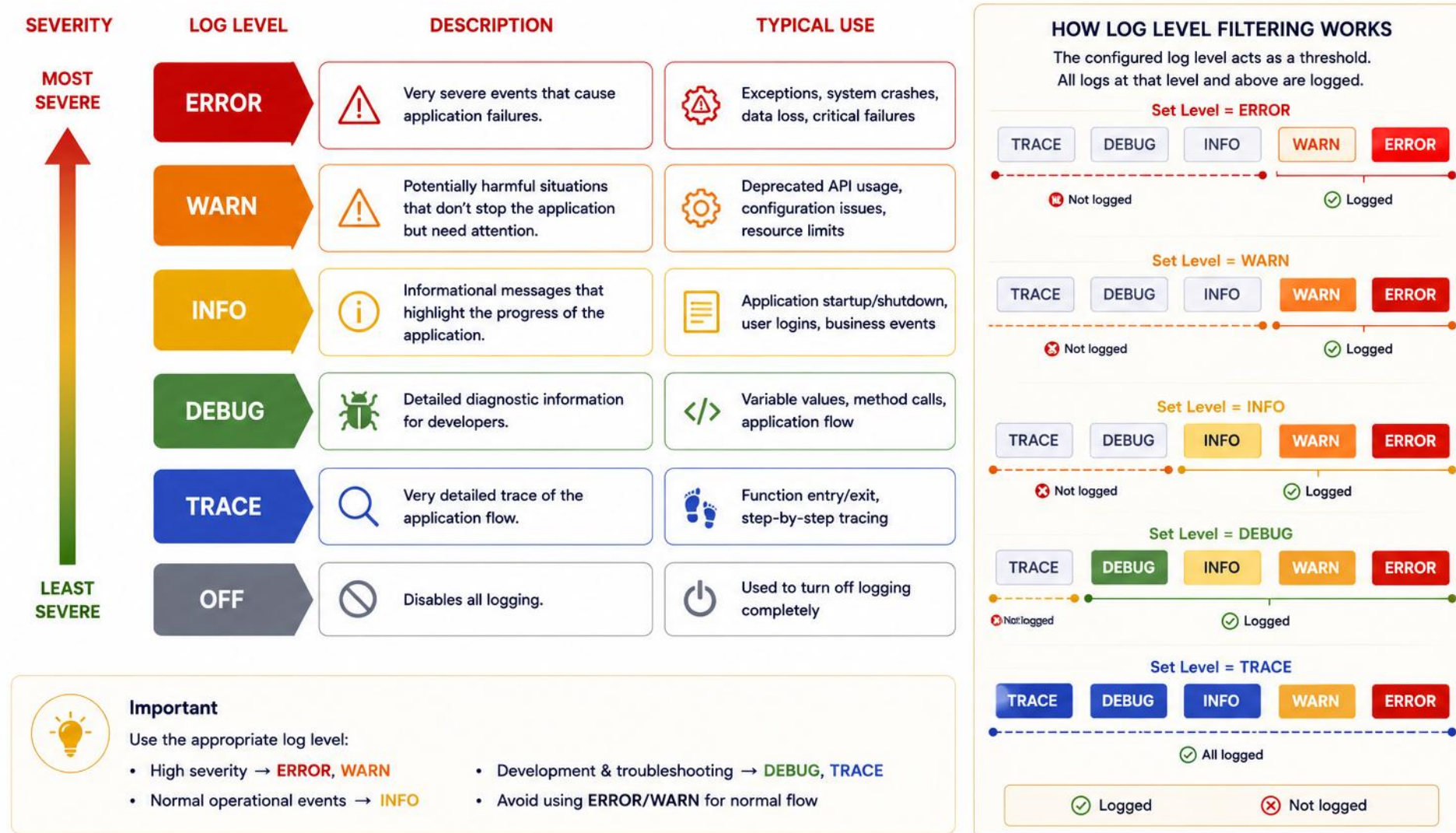
| STEP       | WHAT TO DO                                                                         |
|------------|------------------------------------------------------------------------------------|
| Goal       | Assign the right level — DEBUG / INFO / WARN / ERROR — to four real CRM events     |
| File       | lab20-level-quiz.md (in examples/module-20-exercises/notes/)                       |
| Quiz       | Ravi activation=INFO, Amina transition=WARN, repo failure=ERROR, mapper copy=DEBUG |
| Prod habit | One sentence: DEBUG stays off by default in production profiles                    |
| Your event | Add one CRM event of your own and defend the level you assign it                   |
| Reference  | exercises/exercise-01-level-quiz.md                                                |

```
$ cat lab20-level-quiz.md
Activate success (Ravi)           -> INFO
Illegal transition (Amina)        -> WARN
Unexpected repository failure     -> ERROR
Entering mapper field copy        -> DEBUG
```

**TRY IT NOW** Complete Exercise 1 before continuing — see exercises/exercise-01-level-quiz.md.

# LOG LEVELS EXPLAINED

Log levels define the severity and importance of log events.  
They help control what gets logged and what gets ignored.





# STRUCTURED LOGGING

*Structured logging records events as named fields. JSON is a common transport format because collectors and search platforms can parse it reliably.*

## EXAMPLE (STRUCTURED EVENT)

```
{
  "timestamp": "2025-05-11T10:15:30.520Z",
  "level": "INFO",
  "logger": "OrderService",
  "event": "order.created",
  "operation": "create_order",
  "message": "Order placed",
  "correlationId": "3f9c2a6e7b4dd4e1a",
  "orderId": 10245,
  "outcome": "success"
}
```

## AVOID DUPLICATING FIELDS IN THE MESSAGE

```
// A structured event should not rely on parsing the message:
log.atInfo()
  .addKeyValue("event", "order.created")
  .addKeyValue("orderId", orderId)
  .addKeyValue("outcome", "success")
  .log("Order created");
// Otherwise: use MDC or a structured encoder
```

- Use safe identifiers: an internal non-sensitive subject id where approved, a pseudonymous id, or customer ID only when allowed — never raw email, phone, name, session token, or credential. The lab's zero-PII objective should drive this theory.
- Checklist: stable event name, outcome, reason code, duration, service/environment, correlation or trace context. Common uses: microservices tracing, monitoring & alerting, audit/compliance trails.
- Event naming example: {"event": "customer.created", "operation": "create\_customer", "customerId": "CUS-1001", "outcome": "success"}

**KEY TAKEAWAY** Free-form text becomes difficult to query consistently at scale — structured fields improve filtering, aggregation, and automated analysis.

# TRY IT YOURSELF — EXERCISE 2: REWRITE UNSAFE LOGS

*Reinforces: never logging raw PII — rewrite unsafe log lines into safe, structured ones.*

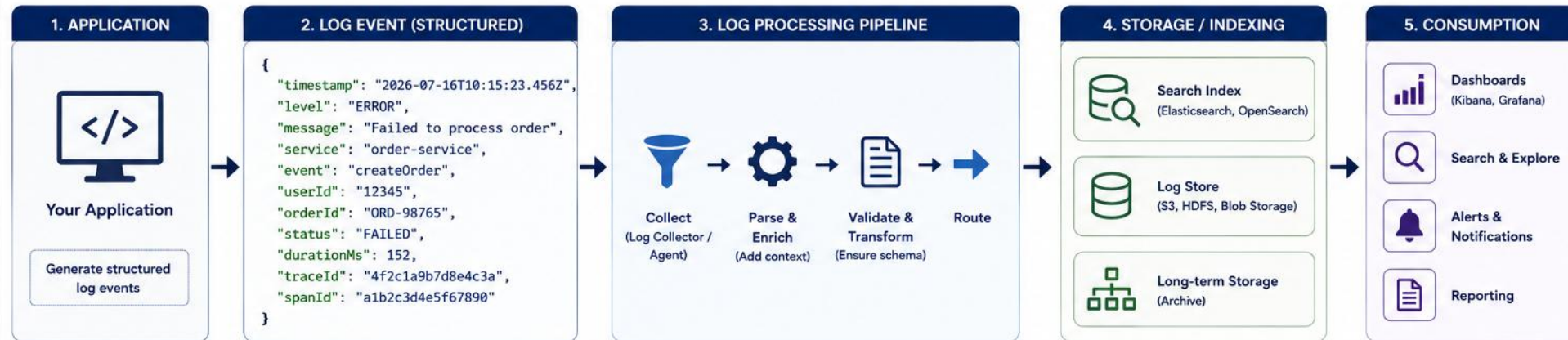
| STEP         | WHAT TO DO                                                                     |
|--------------|--------------------------------------------------------------------------------|
| Goal         | Turn an unsafe Customer log line into a safe id + status + correlation line    |
| File         | lab20-safe-logs.md (in examples/module-20-exercises/notes/)                    |
| Unsafe       | log.info("Customer {}", customer) — toString() may leak email/phone if present |
| Safe (Amina) | customerId=CUS-1001 status=ACTIVE correlation=lab-request-001                  |
| Safe (Ravi)  | Write the same safe pattern for CUS-1002 activating to PROSPECT                |
| Reference    | exercises/exercise-02-rewrite-unsafe-logs.md                                   |

```
$ cat lab20-safe-logs.md
log.info("Customer {}", customer);    // BEFORE: unsafe
log.info("customerId={} status={} corr={}", // AFTER: safe
        "CUS-1001", "ACTIVE", "lab-request-001");
```

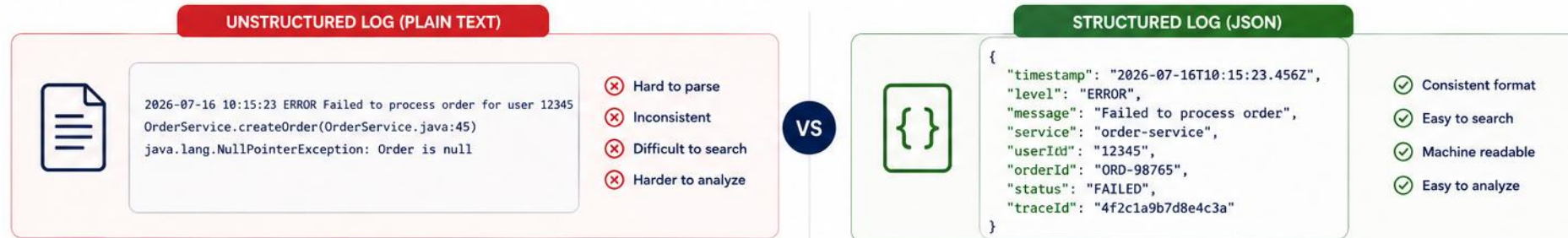
**TRY IT NOW** Complete Exercise 2 before continuing — see exercises/exercise-02-rewrite-unsafe-logs.md.

# STRUCTURED LOGGING

Logging events in a consistent, machine-readable format (key-value pairs) that is easy to search, filter, and analyze.



## UNSTRUCTURED vs STRUCTURED LOG EXAMPLE



## COMMON STRUCTURED LOG FIELDS

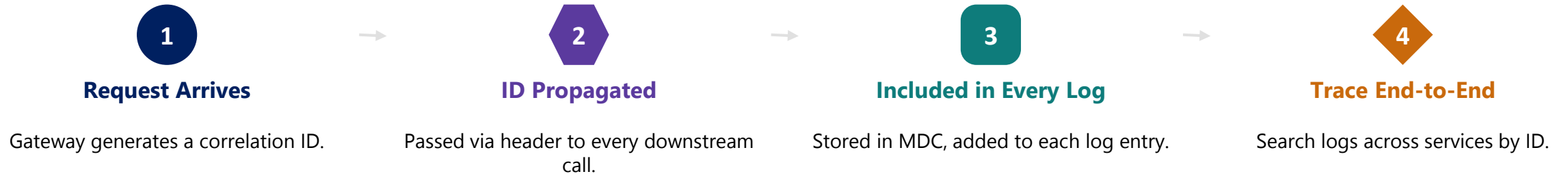


## COMMON STRUCTURED LOG FORMATS



# CORRELATION IDS

*A correlation ID lets you trace a single request across every service and log entry it touches.*



## EXAMPLE (MDC — VALIDATED & PROPAGATED)

```
String incoming = request.getHeader("X-Correlation-Id");
String correlationId = isValid(incoming) ? incoming : newId();
try {
    MDC.put("correlationId", correlationId);
    response.setHeader("X-Correlation-Id", correlationId);
    log.info("Processing order request");
    downstream.propagate(correlationId);
} finally {
    MDC.remove("correlationId");
}
```

*Use an opaque, validated identifier format defined by the organization — not just UUIDs (W3C trace IDs, ULIDs, and other opaque IDs are also used). Do not encode personal or security-sensitive data in it.*

*Ensure context propagation is supported when work moves across threads or reactive boundaries (thread pools, `CompletableFuture`, reactive pipelines, virtual threads). Always clear or restore MDC to prevent context leakage.*

*Correlation ID is application-level; trace ID belongs to distributed tracing (Module 21) — they may coexist, but a correlation ID in logs is not distributed tracing.*

**KEY TAKEAWAY** Correlation IDs connect the dots across distributed systems — without them, tracing a request is guesswork.

# TRY IT YOURSELF — EXERCISE 3: MDC LIFECYCLE

*Reinforces: the MDC put → use → clear lifecycle behind every correlation ID you just saw.*

| STEP      | WHAT TO DO                                                                  |
|-----------|-----------------------------------------------------------------------------|
| Goal      | Sketch the put → use → clear MDC lifecycle for one request's correlation ID |
| File      | lab20-mdc-lifecycle.md (in examples/module-20-exercises/notes/)             |
| Put       | On request entry: MDC.put("correlationId", "lab-request-001")               |
| Use       | Every downstream log line auto-includes it via the logback pattern          |
| Clear     | finally { MDC.clear(); } — always, even when the request fails              |
| Reference | exercises/exercise-03-mdc-lifecycle.md                                      |

```
$ cat lab20-mdc-lifecycle.md
try {
    MDC.put("correlationId", "lab-request-001");
    log.info("Processing request"); // corr auto-included
} finally { MDC.clear(); } // never skip this
```

**TRY IT NOW** Complete Exercise 3 before continuing — see exercises/exercise-03-mdc-lifecycle.md.



# TRY IT YOURSELF — EXERCISE 4: CLEAR MDC IN FINALLY

*Reinforces: always clearing MDC in a finally block so context never leaks across requests.*

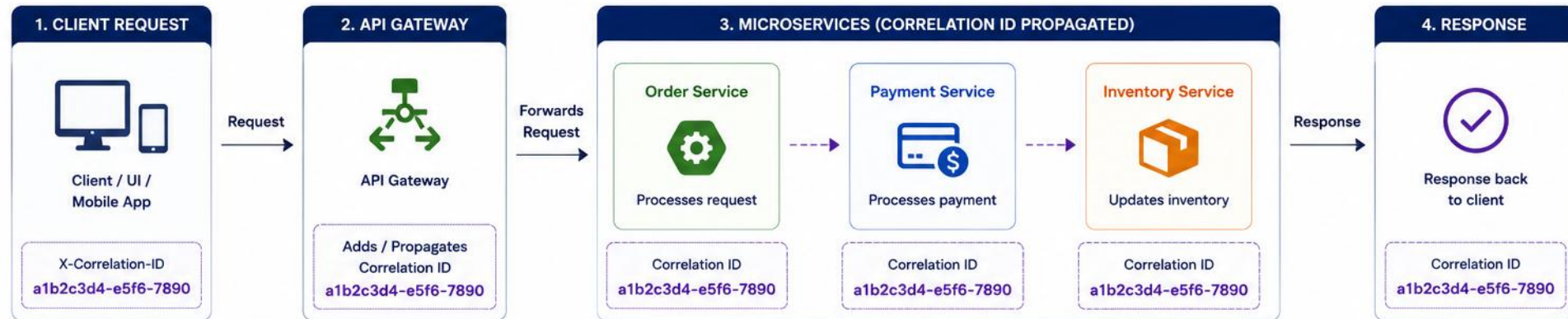
| STEP      | WHAT TO DO                                                                     |
|-----------|--------------------------------------------------------------------------------|
| Goal      | Document why an uncleared MDC key corrupts the next request on a reused thread |
| File      | lab20-mdc-clear.md (in examples/module-20-exercises/notes/)                    |
| Bug       | Request A sets correlation=lab-request-001 and never clears it                 |
| Leak      | Request B reuses the same pooled thread and inherits A's correlation ID        |
| Fix       | A filter/interceptor clears MDC in finally — every request, even on exceptions |
| Reference | exercises/exercise-04-clear-mdc-finally.md                                     |

```
$ cat lab20-mdc-clear.md
Request A: MDC.put("correlationId", "lab-request-001")
// no clear() -> thread returns to pool still holding it
Request B (same thread): MDC.get("correlationId") -> leaked!
// Fix: filter/interceptor always runs MDC.clear() in finally
```

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-clear-mdc-finally.md.

# CORRELATION IDs EXAMPLE

Correlation ID is a unique ID assigned to a request and included in logs across all services.  
It helps trace the full request flow end-to-end.



The same Correlation ID (`a1b2c3d4-e5f6-7890`) is included in every log entry, allowing you to trace the entire request across multiple services and components.

## 5. LOGS WITH CORRELATION ID

| Timestamp               | Service           | Level | Message                                       | Correlation ID                  |
|-------------------------|-------------------|-------|-----------------------------------------------|---------------------------------|
| 2026-07-16 10:15:23.456 | API Gateway       | INFO  | Request received: POST /api/orders            | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:23.457 | Order Service     | INFO  | Order creation started. orderId=ORD-98765     | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:23.789 | Payment Service   | INFO  | Payment initiated. amount=129.99              | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:24.212 | Payment Service   | INFO  | Payment successful. txnId=TXN-555123          | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:24.518 | Inventory Service | INFO  | Inventory updated. orderId=ORD-98765          | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:24.900 | Order Service     | INFO  | Order created successfully. orderId=ORD-98765 | <code>a1b2c3d4-e5f6-7890</code> |
| 2026-07-16 10:15:25.001 | API Gateway       | INFO  | Response sent: 201 Created                    | <code>a1b2c3d4-e5f6-7890</code> |

## HOW IT HELPS



Track a request end-to-end across all services



Easily identify where a failure occurred



Measure latency at each step



Improve debugging and monitoring

Always pass the Correlation ID in headers and include it in logs.

# LOGGING REQUEST FLOWS

*Log incoming requests and outgoing responses with enough context to reconstruct what happened.*

## WHAT TO INCLUDE

- HTTP method, normalized route (e.g. /customers/{id}), and status code — remove or allowlist query parameters.
- Response time / latency.
- Correlation ID for every entry.
- Never log sensitive fields — mask passwords and tokens.

## EXAMPLE

```
log.info("{} {} -> {} ({} ms)",
        method, uri, status, elapsedMs);
// Business decision (structured):
{"event":"customer.activation",
 "outcome":"rejected",
 "reasonCode":"INVALID_STATUS_TRANSITION",
 "customerId":"CUS-1001"}
```

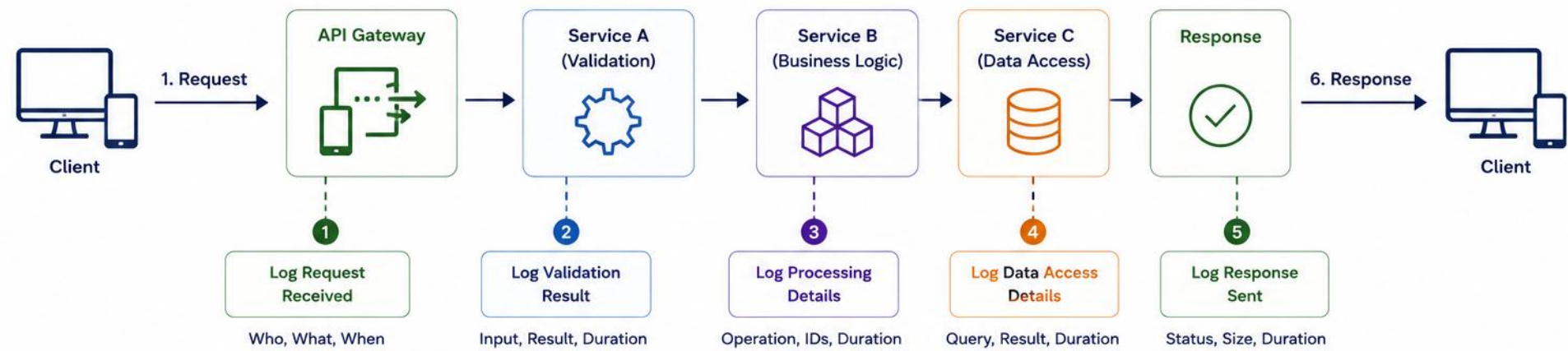
*Also: use database-driver/framework diagnostics at DEBUG when troubleshooting; use metrics for latency and error rate; log business-level persistence outcomes; never log bind values containing secrets or PII; avoid N+1 troubleshooting through permanent high-volume logs.*

*Access logs (transport-level request summary, often already recorded by web servers/gateways/service meshes) differ from application/business logs (business events and decisions), audit logs (security/compliance actions), and trace spans (distributed timing/causality). Avoid duplicating at the application level what access logs already capture.*

**KEY TAKEAWAY** Consistent request/response logging turns every API call into a searchable, traceable record.

# LOGGING REQUEST FLOWS

Log at key points of a request's lifecycle to understand how it moves through the system.



## EXAMPLE LOG ENTRIES

|   | Timestamp               | Correlation ID     | Service / Component | Log Level | Message                    | Key Details                                     |
|---|-------------------------|--------------------|---------------------|-----------|----------------------------|-------------------------------------------------|
| 1 | 2026-07-16 10:15:23.456 | a1b2c3d4-e5f6-7890 | API Gateway         | INFO      | Request received           | method=POST, path=/api/orders, client=10.0.0.5  |
| 2 | 2026-07-16 10:15:23.462 | a1b2c3d4-e5f6-7890 | Service A           | INFO      | Validation successful      | userId=12345, payloadSize=1.2KB, durationMs=6   |
| 3 | 2026-07-16 10:15:23.470 | a1b2c3d4-e5f6-7890 | Service B           | INFO      | Order created              | orderId=ORD-98765, amount=129.99, durationMs=18 |
| 4 | 2026-07-16 10:15:23.487 | a1b2c3d4-e5f6-7890 | Service C           | INFO      | Database update successful | table=orders, rows=1, durationMs=11             |
| 5 | 2026-07-16 10:15:23.505 | a1b2c3d4-e5f6-7890 | API Gateway         | INFO      | Response sent              | status=201, responseSize=512B, durationMs=49    |

## KEY LOGGING POINTS IN A REQUEST FLOW



**Request Received**  
Capture who made the request and what was requested.



**Validation**  
Log validation outcome and important inputs.



**Processing**  
Log business actions, IDs, and processing duration.



**Data Access**  
Log database calls, queries, and results.



**Response Sent**  
Log the response status, size, and total time taken.



Include Correlation ID in every log to trace the flow end-to-end across all services.

**Common Fields:**  
timestamp, correlationId, service, level, message, duration, status



# LOGGING EXCEPTIONS

*Log exceptions with the full stack trace and enough context to find the root cause quickly.*

## EXAMPLE

```
// Lower layer: add typed context, do NOT log here
catch (OrderException ex) {
    throw new OrderProcessingException(orderId, ex);
}
// Boundary: log exactly once
@ExceptionHandler(OrderProcessingException.class)
void handle(OrderProcessingException ex) {
    log.error("Order failed id={}", orderId, ex);
}
```

## BEST PRACTICES

- Include the exception object when stack-trace diagnostics are useful and permitted. Avoid stack-trace noise for expected failures.
- Include the correlation ID and a safe identifier for the affected resource, following data-classification rules — don't expose sensitive information.
- Use ERROR level only for failures that need investigation or cause request/system failure.
- Log an exception at the boundary where sufficient context exists and where it will not be logged again. Lower layers should usually add typed context and rethrow without duplicate logging.

| EXCEPTION CATEGORY                   |
|--------------------------------------|
| Expected validation/domain rejection |
| Security event                       |
| Recoverable dependency issue         |
| Unexpected defect                    |
| Duplicate log from upstream boundary |

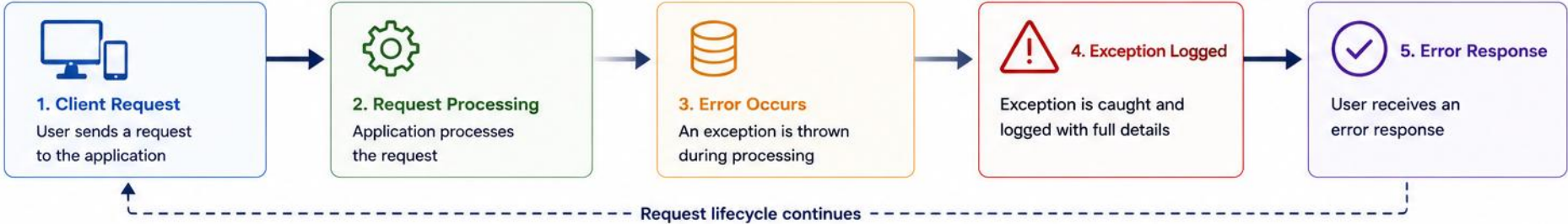
| TYPICAL TREATMENT                                   |
|-----------------------------------------------------|
| Safe response, metric; usually no ERROR stack trace |
| Controlled audit record                             |
| WARN or ERROR based on impact                       |
| ERROR with stack trace                              |
| Avoid                                               |

**KEY TAKEAWAY** A stack trace without context is half a clue — pair it with the correlation ID and relevant business data.



# LOGGING EXCEPTIONS

Log exceptions with meaningful details to quickly identify, diagnose, and resolve issues.

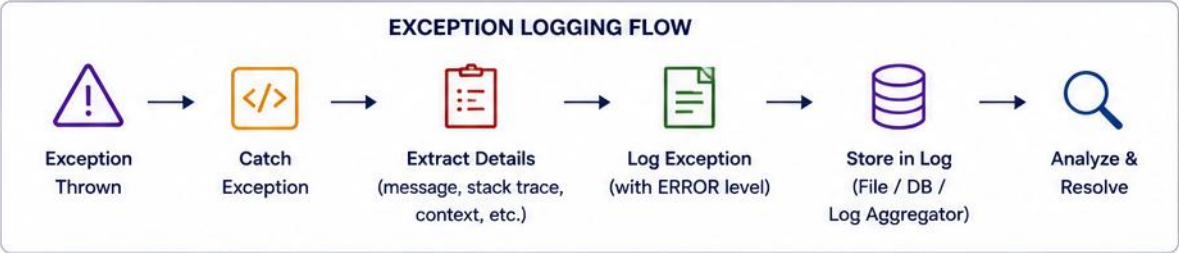


EXAMPLE: EXCEPTION LOG ENTRY

```
{
  "timestamp": "2026-07-16T10:15:23.456Z",
  "level": "ERROR",
  "thread": "http-nio-8080-exec-7",
  "logger": "com.example.order.service.OrderService",
  "message": "Failed to create order",
  "exception": "java.sql.SQLIntegrityConstraintViolationException",
  "exceptionMessage": "Duplicate entry 'ORD-98765' for key 'orders.PRIMARY'",
  "stackTrace": "java.sql.SQLIntegrityConstraintViolationException: Duplicate entry 'ORD-98765' for key 'orders.PRIMARY'\n\tat com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:129)\n\tat com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)\n\tat com.mysql.cj.jdbc.ClientPreparedStatement.executeInternal(ClientPreparedStatement.java:953)\n\tat ... 23 more",
  "correlationId": "a1b2c3d4-e5f6-7890",
  "userId": "12345",
  "requestId": "req-8f7e6d5c",
  "service": "order-service",
  "environment": "production"
}
```

WHAT SHOULD BE LOGGED

|  |                |                                           |
|--|----------------|-------------------------------------------|
|  | When           | Timestamp of the exception                |
|  | What           | Clear error message                       |
|  | Exception Type | Fully qualified exception class           |
|  | Stack Trace    | Full stack trace for root cause           |
|  | Context        | User ID, Request ID, Correlation ID, etc. |
|  | Where          | Service/Component where error occurred    |
|  | Environment    | Environment (dev, test, prod, etc.)       |



- WHY IT MATTERS
- Faster root cause analysis
  - Easier monitoring and alerting
  - Better reliability and system stability
  - Useful for audit and compliance

# CENTRALIZED OPERATIONS AND GOVERNANCE

*Collecting, storing, and visualizing logs from all services in one platform provides a major operational signal that complements metrics and traces.*



## Collection

Ship logs from every service to a central pipeline.



## Storage & Indexing

Index log data so it stays fast to query at scale.



## Visualization

Turn raw log data into dashboards teams can read.



## Search Across Services

Find every entry for a request across all services at once.



## Alerting

Notify the team automatically when error patterns spike.



## Trend Analysis

Spot recurring issues and usage patterns over time.

*Typical stack: Fluent Bit (agent) → Logstash (pipeline) → Elasticsearch (index) → Kibana (visualize) → Alertmanager/Slack (notify) — illustrative; tooling varies by organization. Alert on actionable patterns and sustained conditions rather than every ERROR event. Better alert sources: error-rate metrics, aggregated log patterns, SLO burn rates, dependency failure rates, security rules.*

*Retention & governance: hot searchable retention, archive retention, deletion requirements, legal hold, cost controls, access control, and regional data residency.*

**KEY TAKEAWAY** Centralized logging provides observability at scale — searching one server's log files doesn't work in production.

# SHIPPING LOGS TO CENTRALIZED PLATFORMS

*Emit structured JSON to stdout — a collector like Fluent Bit or Filebeat ships it onward.*

## EXAMPLE: logback-spring.xml

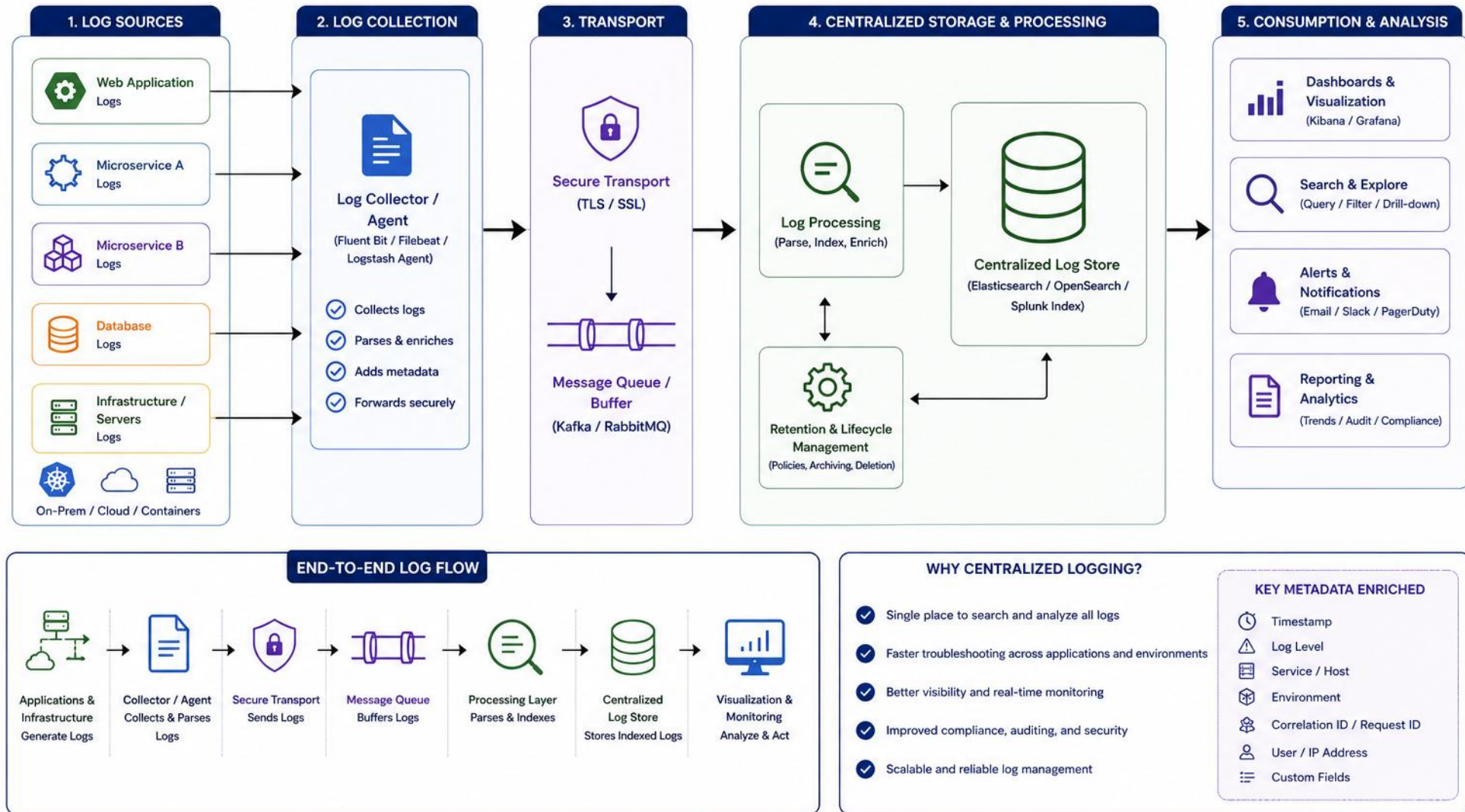
```
<appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    <includeMdcKeyName>correlationId</includeMdcKeyName>
    <customFields>{"service":"order-service"}</customFields>
  </encoder>
</appender>
<root level="INFO">
  <appender-ref ref="JSON_CONSOLE"/>
</root>
```

*Containers and Kubernetes capture stdout automatically, so a node-level agent (Fluent Bit, Filebeat) tails it and forwards structured JSON to the pipeline — no custom SocketAppender or DBAppender code needed in the application. The correlationId set via MDC flows into every JSON line automatically. Add the logstash-logback-encoder dependency to enable this encoder.*

**KEY TAKEAWAY** Structured stdout plus a collector sidecar is the standard, code-free path into a centralized logging platform.

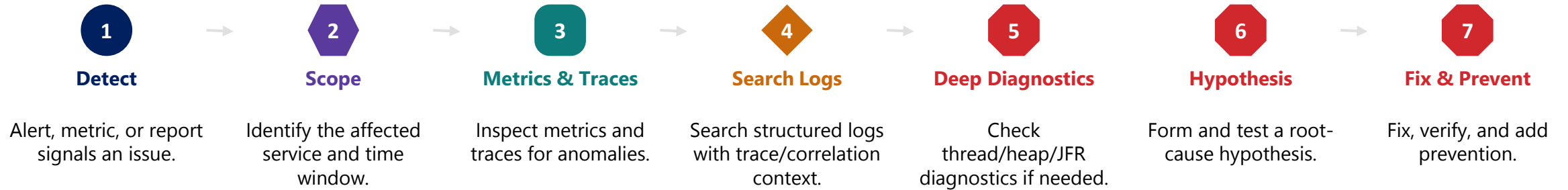
# CENTRALIZED LOGGING ARCHITECTURE

Collect logs from all applications and services in one place for unified storage, search, monitoring, and analysis.



# PRODUCTION DIAGNOSTICS

*Logs are your primary tool for troubleshooting, root cause analysis, and understanding system health in production.*



## KEY PRACTICES

- Design logs around known operational questions while preserving enough safe context for investigation. Do not collect data merely because it might become useful.

*Also track availability, performance, resource, error, and business metrics alongside logs — and always feed lessons learned back in.*

**KEY TAKEAWAY** Design logs around known operational questions, not just to react to outages — diagnostics starts long before an incident.



# CHANGING LOG LEVELS AT RUNTIME

*Raise one logger's verbosity mid-incident, then revert it — without a redeploy.*

## EXAMPLE

```
LoggerContext ctx = (LoggerContext)
    LoggerFactory.getILoggerFactory();
Logger target =
    ctx.getLogger("com.example.order.OrderService");
target.setLevel(Level.DEBUG);
log.warn("DEBUG enabled on OrderService (INC-4821)");

// After diagnosing, revert the level:
target.setLevel(Level.INFO);
```

*LoggerContext is Logback-specific; other bindings expose similar runtime APIs. Combine a scoped DEBUG window with the correlation ID from earlier in this module to isolate output to a single request instead of flooding every log line.*

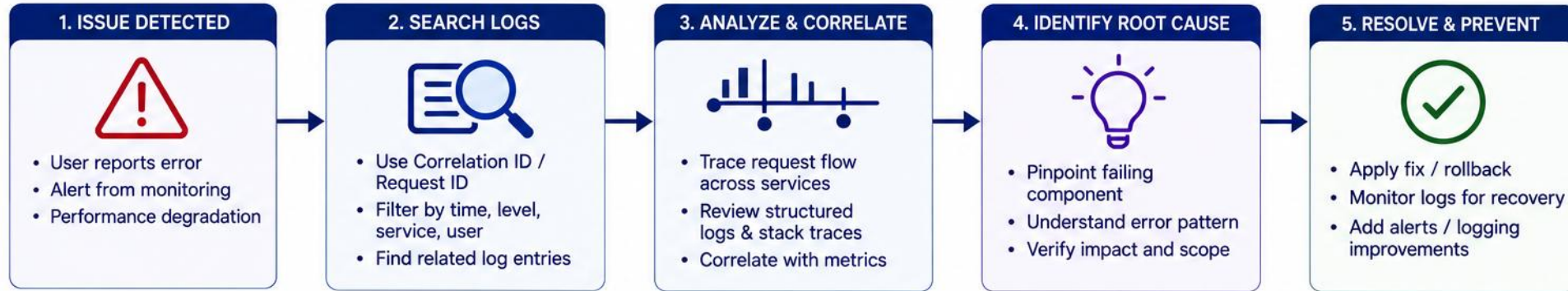
## SAFER IN PRODUCTION

- Prefer Spring Boot Actuator: POST /actuator/loggers/{name} with {"configuredLevel":"DEBUG"} — no custom code or restart needed.
- Scope the change to one class or package, never root — a root DEBUG flood can overwhelm storage and I/O.
- Set a reminder (or automate a TTL) to revert the level once the incident is resolved.
- Log the level change itself, with who/why/when, for the audit trail.

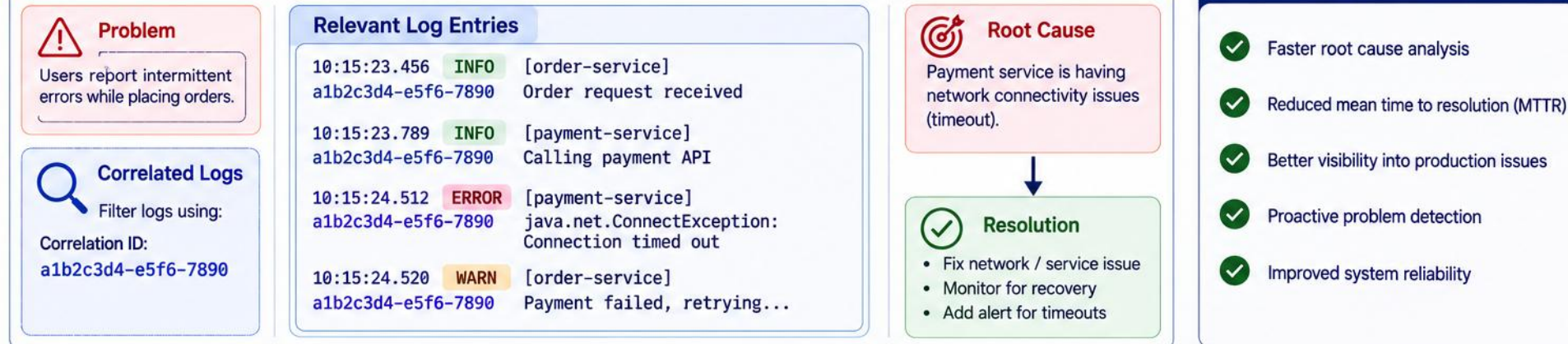
**KEY TAKEAWAY** The fastest diagnostic tool is often the log level you already have — no redeploy required.

# PRODUCTION DIAGNOSTICS

Using logs for fast issue detection and root cause analysis in production environments.



## EXAMPLE: DIAGNOSING A PRODUCTION ISSUE



## KEY BENEFITS

- ✓ Faster root cause analysis
- ✓ Reduced mean time to resolution (MTTR)
- ✓ Better visibility into production issues
- ✓ Proactive problem detection
- ✓ Improved system reliability

## PRODUCTION DIAGNOSTICS BEST PRACTICES



# LOGGING PRACTICES QUICK REFERENCE

*A compact technical reference for five recurring practices — not a repeat of earlier explanations.*

| PRACTICE            | DONE WHEN                                                  | EXAMPLE                                             |
|---------------------|------------------------------------------------------------|-----------------------------------------------------|
| Log with Purpose    | Event is meaningful, not routine noise                     | Avoid logging every method entry/exit               |
| Consistent Format   | Same fields/structure across services                      | Shared schema (JSON or logfmt)                      |
| Include Context     | Timestamp, level, correlation ID, safe identifiers present | Entry is self-explanatory without extra lookups     |
| Never Log Secrets   | Zero passwords, tokens, or raw PII in any field            | Mask/allowlist fields — see hierarchy below         |
| Use the Right Level | Level reflects operational meaning, not habit              | Environment-specific defaults, controlled overrides |

- Also: keep logs concise, watch the performance overhead of logging, plan retention/archival policies, and monitor your logging pipeline itself.
- Sensitive-data hierarchy: (1) do not collect, (2) allowlist safe fields, (3) tokenize/pseudonymize where needed, (4) redact as defense in depth — redaction can fail; partial masking may still expose regulated identifiers.
- Define environment-specific defaults and allow controlled, temporary diagnostic overrides for selected loggers.

**KEY TAKEAWAY** These five practices turn logging from routine noise into a trustworthy operational signal.

# TRY IT YOURSELF — EXERCISE 5: PII CHECKLIST (STARTER)

*Reinforces: the forbidden-vs-allowed field list behind every PII-free log line.*

| STEP       | WHAT TO DO                                                                          |
|------------|-------------------------------------------------------------------------------------|
| Goal       | FILL-IN-THE-BLANK STARTER — complete a forbidden-PII logging checklist, 6 blanks    |
| File       | lab20-pii-todos.md (in examples/module-20-exercises/notes/)                         |
| Fill in    | 3 forbidden fields, 2 allowed ids (CUS-1001 / CUS-1002 + correlation), 1 MDC answer |
| Forbidden  | email, phone, and raw card/national-ID-style values — never in a log                |
| Self-check | Confirm the allowed ids are safe fixtures, never a real name or email               |
| Reference  | exercises/exercise-05-fill-forbidden-pii-todos.md (starter)                         |

```
$ cat lab20-pii-todos.md
Forbidden: _____ // email      Forbidden: _____ // phone
Forbidden: _____ // raw card / national id
Allowed: customerId _____ // CUS-1001 / CUS-1002
Clear MDC in finally? _____ // yes
```

**TRY IT NOW** Complete Exercise 5 (starter) — see exercises/exercise-05-fill-forbidden-pii-todos.md.

# TRY IT YOURSELF — EXERCISE 6: LAB 20 PREP CHECKLIST

*Reinforces: all five pre-lab exercises, together, before Lab 20.*

| STEP      | WHAT TO DO                                                                         |
|-----------|------------------------------------------------------------------------------------|
| Goal      | Confirm all five pre-lab artifacts exist and are filled in before Lab 20           |
| File      | notes/ (5 files, in examples/module-20-exercises/)                                 |
| Checklist | Safe logs, MDC lifecycle, level quiz, PII TODOs, and clear-MDC drill — all present |
| Boundary  | Pre-lab only — full observability metrics wait for Lab 21                          |
| Pass/Fail | Pass only if Exercise 4's PII checklist is genuinely filled in, not blank          |
| Reference | exercises/exercise-06-lab20-prep-checklist.md                                      |

```
$ ls notes/  
lab20-safe-logs.md      lab20-mdc-lifecycle.md      lab20-level-quiz.md  
lab20-pii-todos.md      lab20-mdc-clear.md  
// 5 files, one per exercise (1-5) – all present and filled in  
// ready for Lab 20
```

**TRY IT NOW** Complete Exercise 6 before continuing — see exercises/exercise-06-lab20-prep-checklist.md.



# LAB OVERVIEW – STRUCTURED LOGGING

*Add SLF4J + Logback structured logging with correlation IDs to the Northstar CRM customer service — with zero PII in logs.*

## WHAT YOU WILL BUILD

- A CorrelationFilter that validates/generates the ID, sets MDC + response header, propagates it downstream, and clears it in finally.
- Minimum: consistent key-value (logfmt) fields  
event=/operation=/outcome=/reasonCode=/customerId=/correlationId=/durationMs=. Preferred: JSON encoder with the same named fields.
- Instrumented CustomerService create/get calls — customer IDs only, never PII.

## QUOTE

- "Well-structured logs today become actionable insights tomorrow." Support can search what happened to a request without ever seeing a name, email, or phone number.

## TECH STACK

| ITEM      | VALUE                          |
|-----------|--------------------------------|
| Framework | Spring Boot                    |
| Logging   | SLF4J + Logback                |
| Format    | Pattern layout (JSON optional) |
| Tracing   | MDC correlation ID             |

*Scenario: extend the Customer Management Platform (Lab 19) so support can trace CUS-1001 / CUS-1002 by correlation ID.*

**KEY TAKEAWAY** This lab builds the PII-free, correlation-traceable logging foundation centralized logging platforms depend on.

# LAB TASKS AND DELIVERABLES

*Add structured, PII-free logging with correlation IDs to the Northstar CRM customer service.*

| # | TASK                            | KEY ACTIVITIES                                                              | FOCUS           |
|---|---------------------------------|-----------------------------------------------------------------------------|-----------------|
| 1 | Branch Lab 19 & Confirm Logging | Copy to lab20-crm; confirm SLF4J + Logback binding.                         | Setup           |
| 2 | Configure Structured Pattern    | Build logback-spring.xml with corr/cust/op fields.                          | Structured logs |
| 3 | Add Correlation Filter          | CorrelationFilter puts ID in MDC; clears in finally.                        | Tracing         |
| 4 | Instrument Customer Service     | Log create/get with customerId + op — never PII.                            | Request flows   |
| 5 | Log Validation & Exceptions     | Category-specific levels — not blanket WARN/ERROR (see verification notes). | Diagnostics     |
| 6 | Verify with Automated Test      | CustomerLoggingIT — ListAppender-based; asserts semantically; no PII.       | Verification    |

## DELIVERABLES

- lab20-crm project — structured, PII-free logging.
- logback-spring.xml — logfmt minimum, JSON preferred.
- CorrelationFilter — validate/generate, MDC, header, clear in finally.
- Instrumented CustomerService/Controller logging (no PII).
- CustomerLoggingIT — ListAppender-based, semantic no-PII proof.
- docs/logging.md contract; submit via course portal by due date.

## SUCCESS CRITERIA & VERIFICATION

- Logs show corr=/cust=/op= for CUS-1001 and CUS-1002; CustomerLoggingIT passes; zero PII (names, emails, phones) in any log line; MDC cleared after every request.
- No-PII checks are strong evidence, not proof — layer fixture leak tests + code review.
- MDC-leak test: req A (corr-A) → req B (corr-B) → background log → confirm no corr-A leak.
- One exception, one stack trace — don't double-log; levels are category-specific, not blanket.
- Async propagation (if used): optional advanced exercise — verify cleanup.

**KEY TAKEAWAY** You will have PII-free, correlation-traceable logs that let support search “what happened to request X” with confidence.

# PRODUCTION LOGGING DEFINITION OF DONE

*A checklist for shipping logging that is safe, useful, and consistent in production.*

- 1 Application uses SLF4J rather than backend-specific APIs.
- 2 Exactly one logging provider is configured.
- 3 Events have stable names and structured fields.
- 4 Levels reflect operational meaning.
- 5 Expected client errors do not generate noisy stack traces.
- 6 Unexpected failures are logged once.
- 7 Correlation context is validated, propagated, returned, and cleared.
- 8 Raw URLs, bodies, tokens, names, emails, and phone numbers are not logged.
- 9 Safe identifiers and reason codes are used.
- 10 Request access logs and business logs have distinct purposes.
- 11 Logs are centrally collected.
- 12 Retention and access policies are defined.
- 13 Logging volume and pipeline health are monitored.
- 14 Tests verify important events and privacy constraints.

**KEY TAKEAWAY** Meeting every item on this checklist is what makes logging production-ready — not just functional.

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of logging frameworks and diagnostics.*

## 1. What is SLF4J?

- A. A logging backend implementation
- B. A logging facade/abstraction over logging frameworks**
- C. A centralized log storage system
- D. A Spring Boot annotation

## 3. An API logs the full request URL, including `?email=user@example.com`. What should change?

- A. Nothing — full URLs are fine for debugging
- B. Log the normalized route, not personal query values**
- C. Encrypt the entire URL before logging
- D. Stop logging that endpoint entirely

## 2. Which log level is for unexpected but recoverable issues?

- A. ERROR
- B. WARN**
- C. INFO
- D. TRACE

## 4. Why prefer structured (JSON) logs?

- A. They are shorter to write
- B. They are easy to parse, search, and analyze**
- C. They remove the need for log levels
- D. They only work in development

**KEY TAKEAWAY** SLF4J + Logback, the right log level, and correlation IDs are the foundation of production-ready logging.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions, plus the full answer key.*

## 5. What should NEVER appear in a log entry?

- A. A timestamp
- B. A correlation ID
- C. Passwords or tokens in plain text**
- D. A log level

## 7. What should accompany an exception logged at ERROR?

- A. Nothing else is needed
- B. Exception object, correlation/trace context, safe details**
- C. Only the exception class name
- D. A password reset link

### ANSWER KEY

- 1-B 2-B 3-B 4-B 5-C 6-C 7-B 8-B

## 6. Service and handler both log one exception — what's the main problem?

- A. There is no problem — more logs are more thorough
- B. Each layer should always keep its own audit trail**
- C. Duplicate noise and alerts — log it once at the boundary
- D. It has no real downside

## 8. What is the main goal of production diagnostics?

- A. To avoid writing any logs
- B. To use logs for troubleshooting and root cause analysis**
- C. To disable logging in production
- D. To replace correlation IDs with debuggers

**KEY TAKEAWAY** Great logs lead to great insights, faster fixes, and stronger systems.



## MODULE 20 COMPLETE

# Logging Frameworks and Diagnostics

You can now build a practical, production-ready logging strategy using industry-standard tools and practices.